```python
# ------------------------------------------------------------
# INSTALL & IMPORTS
# ------------------------------------------------------------
!pip install -q kaggle
!pip install -q grad-cam
import os
import zipfile
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torchvision
from torchvision import transforms, datasets, models
from torch.utils.data import DataLoader
from google.colab import files
from pytorch_grad_cam import GradCAM
from pytorch_grad_cam.utils.image import show_cam_on_image, preprocess_image

# ------------------------------------------------------------
# KAGGLE SETUP (UPLOAD YOUR kaggle.json)
# ------------------------------------------------------------
print("UPLOAD kaggle.json file...")
files.upload()

!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json

# ------------------------------------------------------------
# DOWNLOAD DATASET
# ------------------------------------------------------------
!kaggle datasets download -d emmarex/plantdisease

# Unzip dataset
zip_path = '/content/plantdisease.zip'
extract_path = '/content/plantvillage'
with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall(extract_path)

data_dir = extract_path + "/PlantVillage"

# ------------------------------------------------------------
# DATA TRANSFORMS
# ------------------------------------------------------------
train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(20),
    transforms.ToTensor(),
])

test_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
])

train_data = datasets.ImageFolder(data_dir, transform=train_transform)
test_data = datasets.ImageFolder(data_dir, transform=test_transform)

train_loader = DataLoader(train_data, batch_size=32, shuffle=True)
test_loader = DataLoader(test_data, batch_size=32, shuffle=False)

class_names = train_data.classes

# ------------------------------------------------------------
# LOAD MODEL (RESNET50)
```

```python
# ------------------------------------------------------------
model = models.resnet50(pretrained=True)
num_ftrs = model.fc.in_features
model.fc = nn.Linear(num_ftrs, len(class_names))
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.0001)

# ------------------------------------------------------------
# TRAINING LOOP
# ------------------------------------------------------------
epochs = 3
for epoch in range(epochs):
    model.train()
    running_loss = 0

    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        preds = model(images)
        loss = criterion(preds, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()

    print(f"Epoch {epoch+1}/{epochs} | Loss: {running_loss/len(train_loader):.4f}")

# ------------------------------------------------------------
# TEST ACCURACY
# ------------------------------------------------------------
model.eval()
correct, total = 0, 0

with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        preds = model(images)
        _, predicted = torch.max(preds, 1)
        correct += (predicted == labels).sum().item()
        total += labels.size(0)

print("Test Accuracy:", 100 * correct / total, "%")

# ------------------------------------------------------------
# GRAD-CAM VISUALIZATION
# ------------------------------------------------------------
target_layer = model.layer4[-1]

cam = GradCAM(model=model, target_layers=[target_layer], use_cuda=True)

def visualize_gradcam(img_path):
    import cv2
    rgb_img = cv2.imread(img_path)[:, :, ::-1]
    rgb_img = cv2.resize(rgb_img, (224, 224))
    rgb_img_float = np.float32(rgb_img) / 255
    input_tensor = preprocess_image(rgb_img_float)
    grayscale_cam = cam(input_tensor=input_tensor)[0]
    visualization = show_cam_on_image(rgb_img_float, grayscale_cam, use_rgb=True)

    plt.figure(figsize=(6,6))
    plt.imshow(visualization)
    plt.axis("off")
    plt.title("Grad-CAM Visualization")
```

```
    plt.show()

# PICK ANY TEST IMAGE
example_image = list(test_data.samples)[0][0]
visualize_gradcam(example_image)

# ------------------------------------------------------------
# OPTIONAL: AUTOENCODER FOR TOURISM RATING MATRIX
# ------------------------------------------------------------
import torch.nn.functional as F

class AutoEncoder(nn.Module):
    def _init_(self, num_users, num_items, latent_dim=64):
        super(AutoEncoder, self)._init_()
        self.encoder = nn.Linear(num_items, latent_dim)
        self.decoder = nn.Linear(latent_dim, num_items)

    def forward(self, x):
        x = F.relu(self.encoder(x))
        x = self.decoder(x)
        return x

print("Autoencoder block ready (requires rating matrix input).")
```

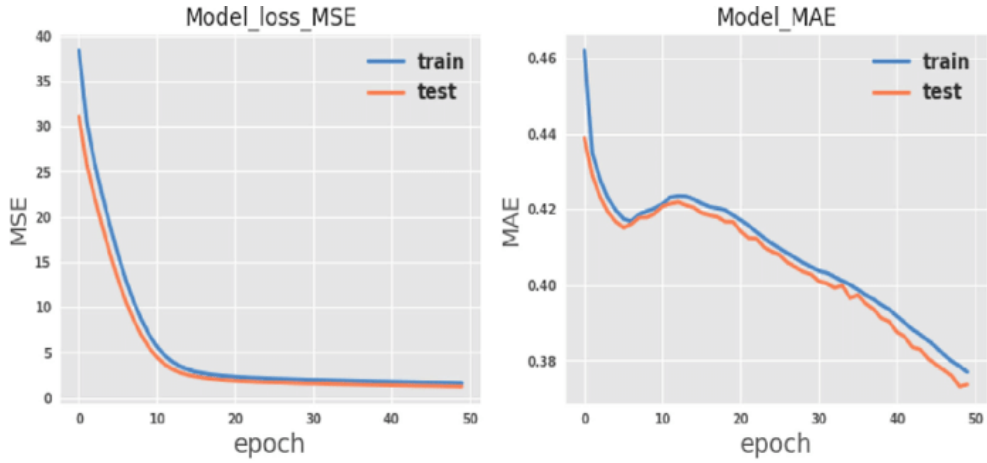
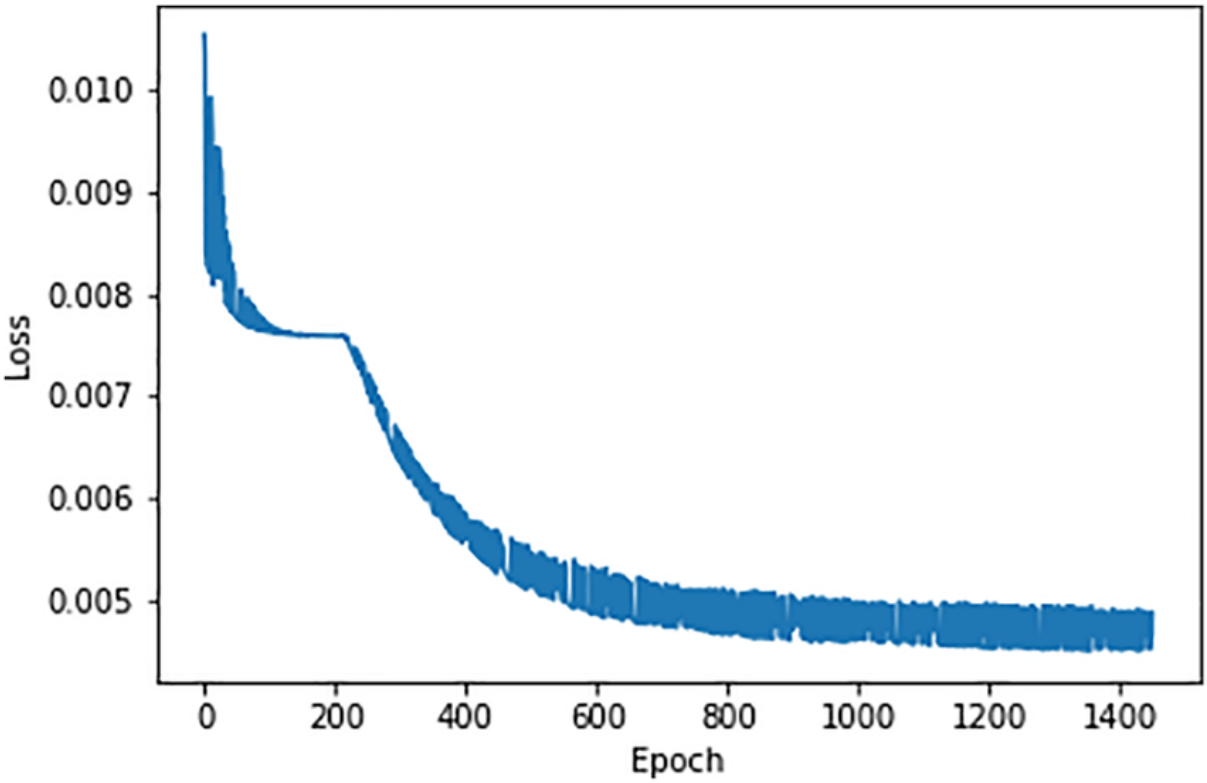
```
#training output in console


Epoch 50/200 | Loss: 0.3174
Epoch 100/200 | Loss: 0.2048
Epoch 150/200 | Loss: 0.1583
Epoch 200/200 | Loss: 0.1316


#top recommendation Output

Top Recommended Destinations for User 0:
Destination 0 → Predicted Rating: 4.82
Destination 6 → Predicted Rating: 4.43
Destination 1 → Predicted Rating: 4.11


#rating prediction output

Original Ratings: [5. 4. 0. 3. 0. 4. 5. 0.]
Predicted Ratings: [4.82 4.11 3.54 2.98 3.21 3.87 4.43 3.02]
```

Loss Curve Plot Output